

=====

CAPSTONE PROJECT

NOTEBOOK 41: FULL-161 END-TO-END INFERENCE PIPELINE

=====

Purpose:

This notebook combines the frozen Stage-1
candidate-150

benchmark hybrid model with the Stage-2
rare-tail fallback

router to produce a working full-161
inference pipeline.

The goal is to:

1. Load aligned expanded hybrid
probabilities
2. Rebuild Stage-1 benchmark predictions
3. Tune Stage-2 rare-tail routing on
validation data

4. Evaluate rare-tail routing on the test split

5. Build deployment-style full-161 outputs:

- primary candidate labels
- rare-tail fallback suggestions

6. Save the final full-161 inference artifacts

=====

In [1]:

```
# =====
# 1. IMPORT LIBRARIES
# =====

import os
import json
import numpy as np
import pandas as pd

SEED = 42
np.random.seed(SEED)

print("Seed set to:", SEED)
```

Seed set to: 42

In [2]:

```
# =====
# 2. LOAD STAGE-1 EXPANDED HYBRID ARTIFACTS
# =====

structured_val_probs =
np.load("../data/processed/structured_on_expanded_audio_candidate150_val_probs.npy"
)
structured_test_probs =
np.load("../data/processed/structured_on_expanded_audio_candidate150_test_probs.npy"
)

audio_val_probs =
```

```

np.load("../data/processed/audio_multilabel_candidate150_expanded_val_probs.npy")
audio_test_probs =
np.load("../data/processed/audio_multilabel_candidate150_expanded_test_probs.npy")

candidate_label_cols = np.load(
    "../data/processed/hybrid_multilabel_candidate150_expanded_label_columns.npy",
    allow_pickle=True
)

val_track_ids =
np.load("../data/processed/structured_on_expanded_audio_candidate150_val_track_ids.
.npy")
test_track_ids =
np.load("../data/processed/structured_on_expanded_audio_candidate150_test_track_ids
.npy")

Y_val_candidate =
np.load("../data/processed/structured_on_expanded_audio_candidate150_y_val.npy")
Y_test_candidate =
np.load("../data/processed/structured_on_expanded_audio_candidate150_y_test.npy")

best_config = {}
with
open("../data/processed/hybrid_multilabel_candidate150_expanded_best_config.txt",
"r") as f:
    for line in f:
        line = line.strip()
        if "=" in line:
            k, v = line.split("=")
            best_config[k.strip()] = float(v.strip())

best_structured_weight = best_config["structured_weight"]
best_audio_weight = best_config["audio_weight"]
best_stage1_threshold = best_config["threshold"]

print("Structured validation probs:", structured_val_probs.shape)
print("Structured test probs:", structured_test_probs.shape)
print("Audio validation probs:", audio_val_probs.shape)
print("Audio test probs:", audio_test_probs.shape)
print("Candidate labels:", len(candidate_label_cols))
print("Validation track IDs:", val_track_ids.shape)
print("Test track IDs:", test_track_ids.shape)
print("Stage-1 best config:", best_config)

```

```

Structured validation probs: (1500, 150)
Structured test probs: (1500, 150)
Audio validation probs: (1500, 150)
Audio test probs: (1500, 150)
Candidate labels: 150
Validation track IDs: (1500,)
Test track IDs: (1500,)
Stage-1 best config: {'structured_weight': 0.1, 'audio_weight': 0.9, 'threshold':
0.2}

```

In [3]:

```
# =====
# 3. LOAD STAGE-2 RARE-TAIL ROUTER ARTIFACTS
# =====

rare_tail_router_df =
pd.read_csv("../data/processed/full161_rare_tail_routing_table.csv")
anchor_to_tail_df =
pd.read_csv("../data/processed/full161_anchor_to_rare_tail_table.csv")
stage2_strategy_df =
pd.read_csv("../data/processed/full161_stage2_execution_strategy.csv")
full_master_df = pd.read_csv("../data/processed/multilabel_full_master_table.csv")
genre_inventory_df = pd.read_csv("../data/processed/full_genre_inventory.csv")

print("Rare-tail router shape:", rare_tail_router_df.shape)
print("Anchor-to-tail table shape:", anchor_to_tail_df.shape)
print("Stage-2 strategy shape:", stage2_strategy_df.shape)
print("Full master shape:", full_master_df.shape)
print("Genre inventory shape:", genre_inventory_df.shape)

display(rare_tail_router_df.head())
display(stage2_strategy_df.head())
```

Rare-tail router shape: (13, 26)
Anchor-to-tail table shape: (13, 10)
Stage-2 strategy shape: (13, 8)
Full master shape: (81574, 170)
Genre inventory shape: (163, 11)

	rare_tail_genre_id	rare_tail_genre_name	parent_id	parent_name	root_genre_id	root_genre_
0	176	Pacific	2	International	2	Interna
1	1060	Tango	46	Latin America	2	Interna
2	465	Musical Theater	20	Spoken	20	Sp
3	189	Talk Radio	65	Radio	20	Sp
4	1032	Turkish	102	Middle East	2	Interna

5 rows × 26 columns

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_name	root_candidate_name	feasi
0	176	Pacific	International	International	
1	1060	Tango	Latin America	International	Extr
2	465	Musical Theater	Spoken	Spoken	Extr
3	189	Talk Radio	Radio	Spoken	
4	1032	Turkish	Middle East	International	

In [4]:

```

# =====
# 4. PREPARE LOOKUPS
# =====

genre_inventory_df["genre_id"] = genre_inventory_df["genre_id"].astype(int)

genre_name_map = dict(zip(genre_inventory_df["genre_id"],
                           genre_inventory_df["genre_name"]))
candidate_label_ids = [int(col.replace("genre_", "")) for col in
                        candidate_label_cols]

candidate_id_to_col = {int(col.replace("genre_", "")): col for col in
                        candidate_label_cols}
candidate_id_to_index = {int(col.replace("genre_", "")): i for i, col in
                          enumerate(candidate_label_cols)}

rare_tail_router_df["rare_tail_genre_id"] =
rare_tail_router_df["rare_tail_genre_id"].astype(int)
rare_tail_router_df["anchor_candidate_id"] = pd.to_numeric(
    rare_tail_router_df["anchor_candidate_id"], errors="coerce"
)
rare_tail_router_df["root_candidate_id"] = pd.to_numeric(
    rare_tail_router_df["root_candidate_id"], errors="coerce"
)

print("Candidate ID lookup size:", len(candidate_id_to_index))

```

```
print("Rare-tail labels in router:",
rare_tail_router_df["rare_tail_genre_id"].nunique())
```

Candidate ID lookup size: 150

Rare-tail labels in router: 13

In [5]:

```
# =====
# 5. BUILD STAGE-1 FUSED PROBABILITIES
# =====

def fuse_probabilities(structured_probs, audio_probs, w_structured, w_audio):
    return (w_structured * structured_probs) + (w_audio * audio_probs)

stage1_val_probs = fuse_probabilities(
    structured_val_probs,
    audio_val_probs,
    best_structured_weight,
    best_audio_weight
)

stage1_test_probs = fuse_probabilities(
    structured_test_probs,
    audio_test_probs,
    best_structured_weight,
    best_audio_weight
)

print("Stage-1 fused validation probs:", stage1_val_probs.shape)
print("Stage-1 fused test probs:", stage1_test_probs.shape)
```

Stage-1 fused validation probs: (1500, 150)

Stage-1 fused test probs: (1500, 150)

In [6]:

```
# =====
# 6. REBUILD STAGE-1 HARD PREDICTIONS
# =====

def decode_global_threshold(prob_matrix, threshold):
    return (prob_matrix >= threshold).astype(np.uint8)

stage1_val_pred = decode_global_threshold(stage1_val_probs, best_stage1_threshold)
stage1_test_pred = decode_global_threshold(stage1_test_probs,
best_stage1_threshold)

print("Stage-1 validation hard predictions:", stage1_val_pred.shape)
print("Stage-1 test hard predictions:", stage1_test_pred.shape)
print("Average Stage-1 predicted labels on validation:",
stage1_val_pred.sum(axis=1).mean())
print("Average Stage-1 predicted labels on test:",
stage1_test_pred.sum(axis=1).mean())
```

Stage-1 validation hard predictions: (1500, 150)
 Stage-1 test hard predictions: (1500, 150)
 Average Stage-1 predicted labels on validation: 4.607333333333333
 Average Stage-1 predicted labels on test: 4.720666666666666

In [7]:

```
# =====
# 7. EXTRACT FULL-161 GROUND TRUTH FOR THE SAME TRACKS
# =====

full_master_indexed = full_master_df.set_index("track_id", drop=False)

full_label_cols = [
    col for col in full_master_df.columns
    if col.startswith("genre_") and col.replace("genre_", "").isdigit()
]

rare_tail_cols = [
    col for col in full_label_cols
    if int(col.replace("genre_", "")) not in candidate_label_ids
]

val_full_df = full_master_indexed.loc[val_track_ids].copy()
test_full_df = full_master_indexed.loc[test_track_ids].copy()

Y_val_rare = val_full_df[rare_tail_cols].values.astype(np.uint8)
Y_test_rare = test_full_df[rare_tail_cols].values.astype(np.uint8)

print("Validation full rows:", val_full_df.shape)
print("Test full rows:", test_full_df.shape)
print("Rare-tail columns:", len(rare_tail_cols))
print("Y_val_rare shape:", Y_val_rare.shape)
print("Y_test_rare shape:", Y_test_rare.shape)
```

Validation full rows: (1500, 170)
 Test full rows: (1500, 170)
 Rare-tail columns: 13
 Y_val_rare shape: (1500, 13)
 Y_test_rare shape: (1500, 13)

In [8]:

```
# =====
# 8. FILTER TO HIERARCHY-FALLBACK LABELS ONLY
# =====

fallback_router_df = rare_tail_router_df[
    rare_tail_router_df["fallback_mode"] == "Hierarchy-triggered fallback"
].copy().reset_index(drop=True)

fallback_router_df["anchor_candidate_id"] =
fallback_router_df["anchor_candidate_id"].astype(int)
```

```
fallback_rare_ids = fallback_router_df["rare_tail_genre_id"].astype(int).tolist()
fallback_rare_cols = [f"genre_{gid}" for gid in fallback_rare_ids]

Y_val_rare_fallback = val_full_df[fallback_rare_cols].values.astype(np.uint8)
Y_test_rare_fallback = test_full_df[fallback_rare_cols].values.astype(np.uint8)

print("Hierarchy-fallback rare-tail labels:", len(fallback_rare_ids))
print("Fallback rare-tail IDs:", fallback_rare_ids)
print("Y_val_rare_fallback shape:", Y_val_rare_fallback.shape)
print("Y_test_rare_fallback shape:", Y_test_rare_fallback.shape)
display(fallback_router_df)
```

Hierarchy-fallback rare-tail labels: 10

Fallback rare-tail IDs: [176, 1060, 465, 189, 1032, 374, 173, 493, 377, 808]

Y_val_rare_fallback shape: (1500, 10)

Y_test_rare_fallback shape: (1500, 10)

	rare_tail_genre_id	rare_tail_genre_name	parent_id	parent_name	root_genre_id	root_genre_
0	176	Pacific	2	International	2	Interna
1	1060	Tango	46	Latin America	2	Interna
2	465	Musical Theater	20	Spoken	20	Sp
3	189	Talk Radio	65	Radio	20	Sp
4	1032	Turkish	102	Middle East	2	Interna
5	374	Banter	20	Spoken	20	Sp
6	173	N. Indian Traditional	86	Indian	2	Interna
7	493	Western Swing	651	Country & Western	9	Co
8	377	Deep Funk	19	Funk	14	Sou
9	808	Salsa	46	Latin America	2	Interna

10 rows × 26 columns



In [9]:

```
# =====
# 9. BUILD RARE-TAIL SUGGESTION SCORES
# =====

def build_rare_tail_scores(stage1_prob_matrix, router_df, candidate_index_map):
    """
    Score rare-tail labels using Stage-1 anchor probabilities and
    training co-occurrence strength.
    """
    score_matrix = np.zeros((stage1_prob_matrix.shape[0], len(router_df)),
dtype=np.float32)
```

```

for j, row in router_df.iterrows():
    anchor_id = int(row["anchor_candidate_id"])
    anchor_idx = candidate_index_map[anchor_id]

    anchor_prob = stage1_prob_matrix[:, anchor_idx]

    p_anchor = row["p_rare_given_anchor"]
    p_root = row["p_rare_given_root"]

    p_anchor = 0.0 if pd.isna(p_anchor) else float(p_anchor)
    p_root = 0.0 if pd.isna(p_root) else float(p_root)

    strength = max(p_anchor, p_root)
    score_matrix[:, j] = anchor_prob * strength

return score_matrix

val_rare_scores = build_rare_tail_scores(stage1_val_probs, fallback_router_df,
candidate_id_to_index)
test_rare_scores = build_rare_tail_scores(stage1_test_probs, fallback_router_df,
candidate_id_to_index)

print("Validation rare-tail score matrix:", val_rare_scores.shape)
print("Test rare-tail score matrix:", test_rare_scores.shape)

```

Validation rare-tail score matrix: (1500, 10)

Test rare-tail score matrix: (1500, 10)

In [10]:

```

# =====
# 10. DEFINE RARE-TAIL ROUTING EVALUATION FUNCTIONS
# =====

def generate_rare_tail_suggestions(
    stage1_probs,
    rare_scores,
    router_df,
    candidate_index_map,
    anchor_trigger_threshold=0.20,
    top_k=3
):
    """
    For each track, only allow rare-tail labels whose anchor candidate
    probability crosses the trigger threshold, then rank by rare-tail score.
    """
    suggestions = []

    for i in range(stage1_probs.shape[0]):
        row_suggestions = []

        for j, row in router_df.iterrows():
            anchor_id = int(row["anchor_candidate_id"])
            anchor_idx = candidate_index_map[anchor_id]
            anchor_prob = float(stage1_probs[i, anchor_idx])

```

```

        if anchor_prob >= anchor_trigger_threshold:
            row_suggestions.append({
                "rare_tail_genre_id": int(row["rare_tail_genre_id"]),
                "rare_tail_genre_name": row["rare_tail_genre_name"],
                "anchor_candidate_id": anchor_id,
                "anchor_candidate_name": row["anchor_candidate_name"],
                "anchor_prob": anchor_prob,
                "rare_tail_score": float(rare_scores[i, j])
            })

        row_suggestions = sorted(
            row_suggestions,
            key=lambda x: (x["rare_tail_score"], x["anchor_prob"]),
            reverse=True
       )[:top_k]

        suggestions.append(row_suggestions)

    return suggestions

def evaluate_rare_tail_routing(y_true_rare, suggestion_lists, fallback_rare_ids,
                             name="Routing"):
    """
    Evaluate whether true rare-tail labels appear in the fallback suggestion list.
    """
    fallback_id_to_index = {gid: i for i, gid in enumerate(fallback_rare_ids)}

    eligible_track_count = 0
    hit_at_1 = 0
    hit_at_3 = 0
    hit_at_5 = 0
    any_suggestion_count = 0

    for i in range(y_true_rare.shape[0]):
        true_idx = np.where(y_true_rare[i] == 1)[0]
        true_ids = [fallback_rare_ids[idx] for idx in true_idx]

        if len(true_ids) == 0:
            continue

        eligible_track_count += 1

        suggested_ids = [x["rare_tail_genre_id"] for x in suggestion_lists[i]]

        if len(suggested_ids) > 0:
            any_suggestion_count += 1

            if any(gid in suggested_ids[:1] for gid in true_ids):
                hit_at_1 += 1

            if any(gid in suggested_ids[:3] for gid in true_ids):
                hit_at_3 += 1

            if any(gid in suggested_ids[:5] for gid in true_ids):
                hit_at_5 += 1

```

```

return {
    "Model": name,
    "Eligible Rare-Tail Tracks": eligible_track_count,
    "Tracks With Any Suggestion": any_suggestion_count,
    "Suggestion Coverage": (any_suggestion_count / eligible_track_count) if
eligible_track_count > 0 else np.nan,
    "Rare-Tail Recall@1": (hit_at_1 / eligible_track_count) if
eligible_track_count > 0 else np.nan,
    "Rare-Tail Recall@3": (hit_at_3 / eligible_track_count) if
eligible_track_count > 0 else np.nan,
    "Rare-Tail Recall@5": (hit_at_5 / eligible_track_count) if
eligible_track_count > 0 else np.nan,
}

```

In [11]:

```

# =====
# 11. TUNE RARE-TAIL ROUTING ON VALIDATION
# =====

routing_results = []

for trigger_threshold in [0.10, 0.15, 0.20, 0.25, 0.30]:
    for top_k in [1, 3, 5]:
        val_suggestions = generate_rare_tail_suggestions(
            stage1_val_probs,
            val_rare_scores,
            fallback_router_df,
            candidate_id_to_index,
            anchor_trigger_threshold=trigger_threshold,
            top_k=top_k
        )

        results = evaluate_rare_tail_routing(
            Y_val_rare_fallback,
            val_suggestions,
            fallback_rare_ids,
            name=f"Routing T={trigger_threshold:.2f} Top-{top_k}"
        )

        results["Anchor Trigger Threshold"] = trigger_threshold
        results["Top-K"] = top_k
        routing_results.append(results)

routing_results_df = pd.DataFrame(routing_results).sort_values(
    ["Rare-Tail Recall@3", "Rare-Tail Recall@1", "Suggestion Coverage"],
    ascending=False
).reset_index(drop=True)

print("Validation rare-tail routing search results:")
display(routing_results_df)

```

Validation rare-tail routing search results:

	Model	Eligible Rare- Tail Tracks	Tracks With Any Suggestion	Suggestion Coverage	Rare-Tail Recall@1	Rare-Tail Recall@3	Rare-Tail Recall@5	Anchor Trigger Threshold	Top- K
0	Routing T=0.10 Top-1	4	4	1.00	0.25	0.25	0.25	0.10	1
1	Routing T=0.10 Top-3	4	4	1.00	0.25	0.25	0.25	0.10	3
2	Routing T=0.10 Top-5	4	4	1.00	0.25	0.25	0.25	0.10	5
3	Routing T=0.15 Top-1	4	4	1.00	0.25	0.25	0.25	0.15	1
4	Routing T=0.15 Top-3	4	4	1.00	0.25	0.25	0.25	0.15	3
5	Routing T=0.15 Top-5	4	4	1.00	0.25	0.25	0.25	0.15	5
6	Routing T=0.20 Top-1	4	3	0.75	0.00	0.00	0.00	0.20	1
7	Routing T=0.20 Top-3	4	3	0.75	0.00	0.00	0.00	0.20	3
8	Routing T=0.20 Top-5	4	3	0.75	0.00	0.00	0.00	0.20	5
9	Routing T=0.25 Top-1	4	3	0.75	0.00	0.00	0.00	0.25	1
10	Routing T=0.25 Top-3	4	3	0.75	0.00	0.00	0.00	0.25	3
11	Routing T=0.25 Top-5	4	3	0.75	0.00	0.00	0.00	0.25	5
12	Routing T=0.30 Top-1	4	3	0.75	0.00	0.00	0.00	0.30	1
13	Routing T=0.30	4	3	0.75	0.00	0.00	0.00	0.30	3

	Model	Eligible Rare-Tail Tracks	Tracks With Any Suggestion	Suggestion Coverage	Rare-Tail Recall@1	Rare-Tail Recall@3	Rare-Tail Recall@5	Anchor Trigger Threshold	Top- K
	Top-3								
14	Routing T=0.30 Top-5	4	3	0.75	0.00	0.00	0.00	0.30	5

In [12]:

```
# =====
# 12. SELECT BEST ROUTING CONFIGURATION
# =====

best_routing_row = routing_results_df.iloc[0].to_dict()

best_anchor_trigger_threshold = float(best_routing_row["Anchor Trigger Threshold"])
best_top_k = int(best_routing_row["Top-K"])

print("Best Stage-2 routing configuration:")
print("Anchor trigger threshold:", best_anchor_trigger_threshold)
print("Top-K rare-tail suggestions:", best_top_k)
print("Best validation routing row:")
print(best_routing_row)
```

Best Stage-2 routing configuration:

Anchor trigger threshold: 0.1

Top-K rare-tail suggestions: 1

Best validation routing row:

```
{'Model': 'Routing T=0.10 Top-1', 'Eligible Rare-Tail Tracks': 4, 'Tracks With Any  
Suggestion': 4, 'Suggestion Coverage': 1.0, 'Rare-Tail Recall@1': 0.25, 'Rare-Tail  
Recall@3': 0.25, 'Rare-Tail Recall@5': 0.25, 'Anchor Trigger Threshold': 0.1, 'Top-  
K': 1}
```

In [13]:

```
# =====
# 13. EVALUATE BEST ROUTING ON TEST
# =====

test_suggestions = generate_rare_tail_suggestions(
    stage1_test_probs,
    test_rare_scores,
    fallback_router_df,
    candidate_id_to_index,
    anchor_trigger_threshold=best_anchor_trigger_threshold,
    top_k=best_top_k
)

test_routing_results = evaluate_rare_tail_routing(
    Y_test_rare_fallback,
```

```

test_suggestions,
fallback_rare_ids,
name="Stage-2 Rare-Tail Router - Test"
)

print("Stage-2 rare-tail routing test results:")
print(test_routing_results)

```

Stage-2 rare-tail routing test results:

```

{'Model': 'Stage-2 Rare-Tail Router - Test', 'Eligible Rare-Tail Tracks': 6, 'Tracks
With Any Suggestion': 6, 'Suggestion Coverage': 1.0, 'Rare-Tail Recall@1':
0.3333333333333333, 'Rare-Tail Recall@3': 0.3333333333333333, 'Rare-Tail Recall@5':
0.3333333333333333}

```

In [14]:

```

# =====
# 14. BUILD DEPLOYMENT-STYLE FULL-161 OUTPUT TABLE
# =====

def ids_to_names_from_binary_row(binary_row, label_cols, genre_map):
    predicted_ids = [
        int(col.replace("genre_", ""))
        for col, val in zip(label_cols, binary_row)
        if int(val) == 1
    ]
    predicted_names = [genre_map.get(gid, str(gid)) for gid in predicted_ids]
    return predicted_ids, predicted_names

deployment_rows = []

for i, track_id in enumerate(test_track_ids):
    stage1_ids, stage1_names = ids_to_names_from_binary_row(
        stage1_test_pred[i],
        candidate_label_cols,
        genre_name_map
    )

    fallback_items = test_suggestions[i]
    fallback_ids = [x["rare_tail_genre_id"] for x in fallback_items]
    fallback_names = [x["rare_tail_genre_name"] for x in fallback_items]
    fallback_scores = [round(float(x["rare_tail_score"]), 6) for x in
        fallback_items]

    inventory_only_candidates = []
    for _, row in rare_tail_router_df[rare_tail_router_df["fallback_mode"] ==
        "Inventory only"].iterrows():
        inventory_only_candidates.append(row["rare_tail_genre_name"])

    deployment_rows.append({
        "track_id": int(track_id),
        "stage1_candidate_label_count": len(stage1_ids),
        "stage1_candidate_label_ids": stage1_ids,
        "stage1_candidate_label_names": stage1_names,
        "stage2_rare_tail_suggestion_count": len(fallback_ids),
        "stage2_rare_tail_suggestion_ids": fallback_ids,

```

```
        "stage2_rare_tail_suggestion_names": fallback_names,  
        "stage2_rare_tail_scores": fallback_scores  
    })  
  
full161_deployment_df = pd.DataFrame(deployment_rows)  
  
print("Full-161 deployment-style output table:")  
display(full161_deployment_df.head(20))
```

Full-161 deployment-style output table:

	track_id	stage1_candidate_label_count	stage1_candidate_label_ids	stage1_candidate_label_n
0	568	6	[10, 12, 15, 25, 38, 85]	[Pop, Rock, Electronic, Experimental
1	982	4	[12, 15, 32, 38]	[Rock, Electronic, Noise, Experimental
2	984	2	[12, 38]	[Rock, Experimental
3	988	5	[12, 15, 25, 32, 38]	[Rock, Electronic, Punk, Noise, Experimental
4	1019	6	[12, 15, 32, 38, 41, 1235]	[Rock, Electronic, Noise, Experimental, Electronic
5	1241	3	[15, 32, 38]	[Electronic, Noise, Experimental
6	1243	6	[1, 12, 15, 32, 38, 41]	[Avant-Garde, Rock, Electronic, Noise, Experimental
7	1247	3	[12, 38, 250]	[Rock, Experimental, Instrumental
8	1252	4	[1, 15, 38, 250]	[Avant-Garde, Electronic, Experimental, Instrumental
9	1260	3	[1, 20, 38]	[Avant-Garde, Spoken Word, Experimental
10	1262	5	[1, 20, 32, 38, 250]	[Avant-Garde, Spoken Word, Experimental, Instrumental
11	1268	4	[1, 15, 32, 38]	[Avant-Garde, Electronic, Noise, Experimental
12	1521	4	[1, 12, 32, 38]	[Avant-Garde, Rock, Noise, Experimental
13	1525	3	[12, 15, 38]	[Rock, Electronic, Experimental
14	1579	3	[1, 30, 38]	[Avant-Garde, Field Recordings, Experimental
15	1583	2	[20, 38]	[Spoken Word, Experimental
16	1589	6	[1, 12, 30, 32, 38, 41]	[Avant-Garde, Rock, Recordings, Noise
17	1633	4	[1, 15, 32, 38]	[Avant-Garde, Electronic, Noise, Experimental
18	1867	3	[12, 15, 38]	[Rock, Electronic, Experimental
19	1871	4	[12, 15, 38, 1235]	[Rock, Electronic, Experimental, Instrumental



In [15]:

```

# =====
# 15. BUILD HUMAN-READABLE DEMO TABLE
# =====

demo_rows = []

for i in range(min(25, len(test_track_ids))):
    tid = int(test_track_ids[i])

    stage1_names = full161_deployment_df.loc[i, "stage1_candidate_label_names"]
    stage2_names = full161_deployment_df.loc[i,
"stage2_rare_tail_suggestion_names"]
    stage2_scores = full161_deployment_df.loc[i, "stage2_rare_tail_scores"]

    true_candidate_ids = [
        int(col.replace("genre_", ""))
        for col in candidate_label_cols
        if int(test_full_df.loc[tid, col]) == 1
    ]
    true_candidate_names = [genre_name_map.get(gid, str(gid)) for gid in
true_candidate_ids]

    true_rare_ids = [
        int(col.replace("genre_", ""))
        for col in fallback_rare_cols
        if int(test_full_df.loc[tid, col]) == 1
    ]
    true_rare_names = [genre_name_map.get(gid, str(gid)) for gid in true_rare_ids]

    demo_rows.append({
        "track_id": tid,
        "true_candidate_labels": true_candidate_names,
        "predicted_candidate_labels": stage1_names,
        "true_rare_tail_labels": true_rare_names,
        "rare_tail_suggestions": stage2_names,
        "rare_tail_suggestion_scores": stage2_scores
    })

full161_demo_df = pd.DataFrame(demo_rows)

print("Full-161 demo table:")
display(full161_demo_df)

```

Full-161 demo table:

	track_id	true_candidate_labels	predicted_candidate_labels	true_rare_tail_labels	rare_tail_sug
0	568	[Rock]	[Pop, Rock, Electronic, Punk, Experimental, Ga...		
1	982	[Field Recordings, Experimental]	[Rock, Electronic, Noise, Experimental]		
2	984	[Field Recordings, Experimental]	[Rock, Experimental]		
3	988	[Field Recordings, Experimental]	[Rock, Electronic, Punk, Noise, Experimental]		
4	1019	[Noise, Experimental]	[Rock, Electronic, Noise, Experimental, Electr...		
5	1241	[Noise, Experimental]	[Electronic, Noise, Experimental]		
6	1243	[Noise, Experimental]	[Avant-Garde, Rock, Electronic, Noise, Experim...		
7	1247	[Noise, Experimental]	[Rock, Experimental, Improv]		
8	1252	[Noise, Experimental]	[Avant-Garde, Electronic, Experimental, Improv]		
9	1260	[Noise, Experimental]	[Avant-Garde, Spoken, Experimental]		
10	1262	[Noise, Experimental]	[Avant-Garde, Spoken, Noise, Experimental, Imp...		
11	1268	[Noise, Experimental]	[Avant-Garde, Electronic, Noise, Experimental]		
12	1521	[Rock, Psych-Rock]	[Avant-Garde, Rock, Noise, Experimental]		
13	1525	[Rock, Psych-Rock]	[Rock, Electronic, Experimental]		
14	1579	[Field Recordings, Experimental]	[Avant-Garde, Field Recordings, Experimental]		
15	1583	[Field Recordings, Experimental]	[Spoken, Experimental]		
16	1589	[Field Recordings, Experimental]	[Avant-Garde, Rock, Field Recordings, Noise, E...		
17	1633	[Noise, Experimental]	[Avant-Garde, Electronic, Noise, Experimental]		
18	1867	[Classical, Electronic]	[Rock, Electronic, Experimental]		
19	1871	[Classical, Electronic]	[Rock, Electronic, Experimental, Instrumental]		

	track_id	true_candidate_labels	predicted_candidate_labels	true_rare_tail_labels	rare_tail_sug
20	1873	[Classical, Electronic]	[International, Rock, Electronic, Hip-Hop, Exp...		[]
21	1875	[Electronic]	[Avant-Garde, Rock, Electronic, Noise, Experim...		[]
22	1879	[Electronic]	[Rock, Electronic, Experimental, Instrumental]		[]
23	1880	[Electronic]	[Electronic, Hip-Hop, Experimental]		[]
24	1881	[Electronic]	[Electronic, Hip-Hop, Experimental]		[]

In [16]:

```
# =====
# 16. BUILD FINAL FULL-161 PIPELINE SUMMARY
# =====

pipeline_summary_rows = [
    {
        "Component": "Stage 1",
        "Status": "Ready",
        "Details": "Expanded hybrid benchmark candidate-150 model is frozen."
    },
    {
        "Component": "Stage 2",
        "Status": "Ready",
        "Details": f"Rare-tail router tuned with trigger threshold
{best_anchor_trigger_threshold} and top-{best_top_k} suggestions."
    },
    {
        "Component": "Direct candidate labels",
        "Status": "Ready",
        "Details": f"{len(candidate_label_cols)} labels predicted directly by the
benchmark model."
    },
    {
        "Component": "Hierarchy-fallback rare-tail labels",
        "Status": "Ready",
        "Details": f"{len(fallback_rare_ids)} rare-tail labels available through
Stage-2 fallback suggestions."
    },
    {
        "Component": "Inventory-only rare-tail labels",
        "Status": "Ready",
        "Details": f"{rare_tail_router_df[rare_tail_router_df['fallback_mode'] ==
'Inventory only'].shape[0]} labels retained as inventory only."
    },
    {
        "Component": "Full-161 pipeline",
        "Status": "Operational design ready",
```

```
        "Details": "Stage-1 direct prediction and Stage-2 fallback suggestion
pipeline is now assembled."
    }
]

full161_pipeline_summary_df = pd.DataFrame(pipeline_summary_rows)

print("Full-161 pipeline summary:")
display(full161_pipeline_summary_df)
```

Full-161 pipeline summary:

	Component	Status	Details
0	Stage 1	Ready	Expanded hybrid benchmark candidate-150 model ...
1	Stage 2	Ready	Rare-tail router tuned with trigger threshold ...
2	Direct candidate labels	Ready	150 labels predicted directly by the benchmark...
3	Hierarchy-fallback rare-tail labels	Ready	10 rare-tail labels available through Stage-2 ...
4	Inventory-only rare-tail labels	Ready	3 labels retained as inventory only.
5	Full-161 pipeline	Operational design ready	Stage-1 direct prediction and Stage-2 fallback...

In [17]:

```
# =====
# 17. SAVE OUTPUTS
# =====

os.makedirs("../data/processed", exist_ok=True)

routing_results_df.to_csv(
    "../data/processed/full161_stage2_validation_routing_search.csv",
    index=False
)

pd.DataFrame([test_routing_results]).to_csv(
    "../data/processed/full161_stage2_test_routing_results.csv",
    index=False
)

full161_deployment_df.to_csv(
    "../data/processed/full161_end_to_end_deployment_table.csv",
    index=False
)

full161_demo_df.to_csv(
```

```

    "../data/processed/full161_end_to_end_demo_table.csv",
    index=False
)

full161_pipeline_summary_df.to_csv(
    "../data/processed/full161_end_to_end_pipeline_summary.csv",
    index=False
)

with open("../data/processed/full161_stage2_best_config.txt", "w") as f:
    f.write(f"anchor_trigger_threshold={best_anchor_trigger_threshold}\n")
    f.write(f"top_k={best_top_k}\n")

print("Saved full-161 end-to-end inference outputs.")

```

Saved full-161 end-to-end inference outputs.

In [18]:

```

# =====
# 18. INTERPRETATION NOTES
# =====

print("1. This notebook combines the frozen Stage-1 benchmark model with the Stage-2 rare-tail router.")
print("2. Candidate-150 labels remain the primary direct predictions.")
print("3. Rare-tail labels are surfaced as hierarchy-triggered fallback suggestions rather than conventional hard predictions.")
print("4. This creates a working full-161 inference pipeline design without forcing unreliable direct training on rare-tail labels.")
print("5. The next step can be a deployment-style single-file inference notebook for new audio inputs.")

```

1. This notebook combines the frozen Stage-1 benchmark model with the Stage-2 rare-tail router.
2. Candidate-150 labels remain the primary direct predictions.
3. Rare-tail labels are surfaced as hierarchy-triggered fallback suggestions rather than conventional hard predictions.
4. This creates a working full-161 inference pipeline design without forcing unreliable direct training on rare-tail labels.
5. The next step can be a deployment-style single-file inference notebook for new audio inputs.